Functions

Definitions

Defining a function

Functions are reusable blocks of code that do a specific task. They take input, process it, and return output.

Syntax:

```
def function_name(parameters):
    # random bullshit go
    return value    # default None
```

In [ ]:
```
def hello_there():
    print("Hello there!")
    return "General Kenobi!"
```

Calling a function

In [ ]:
```
return_val = hello_there()
print(f"return_val: {return_val}")
```

```
Hello there!
return_val: General Kenobi!
```

Docstrings

Docstrings are used to describe what a function does.

Syntax:

```
def function_name(parameters):
    """
```

```
    Description of the function
    """
    # random bullshit go
    return value    # default None
```

In [ ]:
```
def hello_there(name):
    """
    This function takes a name and prints out a greeting like General Grevious would say.
    """
    print(f"General {name}!")
```

In [ ]:
```
# To see the docstring, use the help function
?hello_there
# this only works in the interactive shell
```

```
Signature: hello_there(name)
Docstring: This function takes a name and prints out a greeting like General Grevious would say.
File:      /tmp/ipykernel_8052/3698036903.py
Type:      function
```

In [ ]:
```
help(hello_there)
```

```
Help on function hello_there in module __main__:

hello_there(name)
    This function takes a name and prints out a greeting like General Grevious would say.
```

Parameters

Parameters are used to pass information to functions.

In [ ]:
```
def hello_there(name):
    print(f"Hello there!")
    return f"General {name}!"
```

In [ ]:
```
hello_there("Kenobi")
```

```
Hello there!
```

Out[ ]: 'General Kenobi!'

Parameter types are not declared in Python, so you can pass any type of parameter to a function. An error will be raised if the function is called with the wrong number of arguments, or if some incompatible type for the operations done in the function is passed.

In [ ]:
```
hello_there(146.3)
```

```
Hello there!
```

Out[ ]: 'General 146.3!'

Parameter types can be specified in the function definition, but this is not enforced by Python.

In [ ]:
```
def hello_there(name: str) -> str:
    print(f"Hello there!")
    return f"General {name}!"
```

In [ ]:
```
hello_there(146.3)
```

```
Hello there!
```

Out[ ]: 'General 146.3!'

Default arguments

Default values can be set for parameters, so that the function can be called without passing a value for that parameter.

In [ ]:
```
def hello_there(name="Kenobi"):
    print(f"Hello there!")
    return f"General {name}!"

hello_there()
```

```
Hello there!
```

Out[ ]: 'General Kenobi!'

Order of parameters is important, unless you specify the parameter name when calling the function.

In [ ]:
```python
def hello_there(name, rank):
    print(f"General {name}, {rank}!")

hello_there("Kenobi", "Jedi Master")
hello_there("Jedi Master", "Kenobi")
hello_there(rank="Jedi Master", name="Kenobi")
```

```
General Kenobi, Jedi Master!
General Jedi Master, Kenobi!
General Kenobi, Jedi Master!
```

Default arguments must be at the end of the parameter list.

In [ ]:
```python
def hello_there(name, rank="Jedi Master"):
    print(f"General {name}, {rank}!")

hello_there("Kenobi")
```

```
General Kenobi, Jedi Master!
```

In [ ]:
```python
def hello_there(rank="Jedi Master", name):
    print(f"General {name}, {rank}!")
```

```
  Cell In[27], line 1
    def hello_there(rank="Jedi Master", name):
                                            ^
SyntaxError: parameter without a default follows parameter with a default
```

Positional-only arguments

Positional-only arguments are defined by placing a `/` before the arguments in the function definition.

In [ ]:
```python
def hello_there(name, /):
    print(f"General {name}!")

hello_there("Kenobi")
# hello_there(name="Kenobi")  # This will raise an error
```

```
General Kenobi!
```

Keyword-only arguments

In [ ]:
```python
def hello_there(*, name):
    print(f"General {name}!")

hello_there(name="Kenobi")
# hello_there("Kenobi")  # This will raise an error
```

```
General Kenobi!
```

Combining positional-only and keyword-only arguments

NOTE:

`def foo(a, b, /, *, c, d):` is valid, but `def foo(a, b, *, /, c, d):` is not.

In [ ]:
```python
def hello_there(name, /, *, rank="Jedi Master"):
    # here name is positional only and rank is keyword only
    print(f"General {name}, {rank}!")

hello_there("Kenobi", rank="Jedi Knight")
```

```
General Kenobi, Jedi Knight!
```

*args and **kwargs

`*args` and `**kwargs` allow you to pass a variable number of arguments to a function.

`*args` is used to pass a non-keyworded, variable-length argument list.

In [ ]:
```python
# *args
def hello_there(*args):
    print(f"Hello there!")
    for name in args:
        print(f"General {name}!")

hello_there("Kenobi", "Skywalker")
```

```
Hello there!
General Kenobi!
General Skywalker!
```

`**kwargs` is used to pass a keyworded variable-length argument list.

In [ ]:
```python
# **kwargs

def hello_there(**kwargs):
    print(f"Hello there!")
    for name, rank in kwargs.items():
        print(f"General {name}! {rank}")

hello_there(Kenobi="Jedi Master", Skywalker="Not a Master")
```

```
Hello there!
General Kenobi! Jedi Master
General Skywalker! Not a Master
```

Passing variables to functions

In [ ]:
```python
def hello_there(name):
    print(f"General {name}!")
    return name
```

Passing variables to functions is done by reference, so the function can modify the variable.

In [ ]:
```python
s = "Kenobi"
print(id(s))
print(id(hello_there(s)))
```

```
129555750137904
General Kenobi!
129555750137904
```

For a mutable object, the function can modify the object for the changes to be reflected outside the function. Mutable objects include lists, dictionaries, and sets.

In [ ]:
```python
def change_first_element(l):
    l[0] = 99

l = [1, 2, 3]
change_first_element(l)
print(l)
# here the list is mutable and the function changes the list in place
```

```
[99, 2, 3]
```

For an immutable object, the function can modify the object, but the changes will not be reflected outside the function. Immutable objects include strings, numbers, and tuples.

In [ ]:
```python
def change_name(name):
    name = "Skywalker"

s = "Kenobi"
change_name(s)
print(s)
# here the value of s is not changed because strings are immutable
```

```
Kenobi
```

Id of a variable inside the function is not the same as the id of the variable outside the function.

In [ ]:
```python
def foo(a):
    a = 99  # this will not change the value of b, because a is a local variable
    # the id of a will be changed
    print(id(a))

b = 42
print(id(b))
foo(b)
print(b)
```

```
11755208
11757032
42
```

In [ ]:
```
def foo(a):
    print(id(a))    # this will print the same id as b

b = 42
print(id(b))
foo(b)
print(b)
```

```
11755208
11755208
42
```

So the id changes if the variable is reassigned inside the function.

In [ ]:
```
def foo(l):
    l[0] = 99
    print(id(l))
    l = [4, 5, 6]
    print(id(l))

l = [1, 2, 3]
print(id(l))
foo(l)
```

```
129555739266112
129555739266112
129555749525120
```

In [ ]:
```
def foo(t):
    a = list(t)
    a[0] = 99
    print(id(a))
    return tuple(a)

t = (1, 2, 3)
print(id(t))
t = foo(t)
print(id(t))
```

```
129555687025856
129555687076736
129555739358912
```

Scope of variables

Local variables

These are variables initialized inside a function and are only accessible inside that function.

In [ ]:
```python
def foo():
    a = 99
    print(f"local variable a: {a}")

foo()
# print(f"{a}")   # NameError: name 'a' is not defined
# a is a local variable and is not available outside the function
# it's lifetime is only within the function and
# it is destroyed after the function is executed
```

```
local variable a: 99
```

Global variables

These are variables initialized outside a function and are accessible throughout the program.

In [ ]:
```python
b = 42
def foo():
    print(f"global variable b: {b}")

foo()
# b's lifetime is the entire program
```

```
global variable b: 42
```

In [ ]:
```python
def foo():
    global c
```

```
    c = 99
    print(f"local variable c: {c}")

foo()
print(f"global variable c: {c}")
```

```
local variable c: 99
global variable c: 99
```

Global vs Local variables

In [ ]:
```
def foo():
    c = 99
    print(f"local variable c: {c}")

c = 42
print(f"global variable c: {c}")
foo()
print(f"global variable c: {c}")
```

```
global variable c: 42
local variable c: 99
global variable c: 42
```

The `global` keyword is used to access a global variable from inside a function.

In [ ]:
```
def foo():
    global d
    d += 3
    print(f"local variable d: {d}")

d = 42
print(f"global variable d: {d}")
foo()
print(f"global variable d: {d}")
```

```
global variable d: 42
local variable d: 45
global variable d: 45
```

Nested functions

A nested function is a function defined inside another function.
The outer function can return the inner function, or pass it as an argument to another function.
The inner function can access the variables of the outer function.

In [ ]:
```
def outer():
    e = 42
    print(f"e in outer: {e}")
    def inner():
        print(f"e in inner: {e}")
    inner()

outer()
```

```
e in outer: 42
e in inner: 42
```

NOTE: local variables of the outer function can be accessed by the inner function, but **CANNOT BE MODIFIED**.

To modify a variable of the outer function inside the inner function, we use the `nonlocal` keyword.

`nonlocal` keyword

The `nonlocal` keyword is used to modify a variable of the outer function inside the inner function.
Non-local variables are used in nested functions whose local scope is not defined.
This means that the variable can be neither in the local nor the global scope.

In [ ]:
```
def outer():
    e = 42
    print(f"e in outer: {e}")
    def inner():
        nonlocal e
        e += 3
        print(f"e in inner: {e}")
```

```
    inner()
    print(f"e in outer: {e}")

outer()
```

```
e in outer: 42
e in inner: 45
e in outer: 45
```

If there are multiple nested functions, the `nonlocal` keyword will modify the variable of the nearest enclosing scope.

In [ ]:
```
def outer():
    e = 42
    def inner():
        e = 99
        def inner_inner():
            nonlocal e
            e += 3
            print(f"e in inner_inner: {e}")
        inner_inner()
        print(f"e in inner: {e}")
    inner()
    print(f"e in outer: {e}")

outer()
```

```
e in inner_inner: 102
e in inner: 102
e in outer: 42
```

Scope of functions

Accessing nested functions from outside

In [ ]:
```
def outer():
    def inner():
        print("hello there!")
    global exposed_inner
    exposed_inner = inner
```

```
outer()
exposed_inner()
```

```
hello there!
```

In [ ]:
```
def outer():
    def inner():
        print("hello there!")
    def another_inner():
        print("General Kenobi!")
    return inner, another_inner

exposed_inner, exposed_another_inner = outer()
exposed_inner()
exposed_another_inner()
```

```
hello there!
General Kenobi!
```

Higher-order functions

In [ ]:
```
# higher order functions
def foo(m, f):
    return f(m)

def hello_there(name):
    print(f"General {name}!")

foo("Kenobi", hello_there)
```

```
General Kenobi!
```